Room Config Builder

(Deployment Configurator)

Operation Guide · ui_schema.json & key_map.json Setup

How the application works, and how to configure the GUI schema and legacy key mapping for both System settings and Devices

Contents

1. What the program is

The **Room Config Builder** (window title "Deployment Configurator") is a Flutter desktop application that creates, edits, migrates, and deploys the `config.json` file that drives an Extron control-processor room system. Instead of hand-editing raw JSON, an installer works through a set of tabs that render the right editor for each setting, then exports or pushes the finished file straight to the processor over SFTP.

The important design idea is that the application ships with almost no hard-coded knowledge of your config keys. Two external JSON files — `ui_schema.json` and `key_map.json` — tell the program how to *display* each key and how to *translate* legacy key names. You can add a brand-new config property and give it a proper editor with a label, a description, and a dropdown just by editing `ui_schema.json` and pressing **Reload Schema** — no rebuild required.

1.1 The five tabs

Tab	Purpose
Wizard	**Room identity (building, room number, full room name) and hardware quantities. Setting a device count here generates or removes that device's blocks and its detail tabs. Picking a building stores its building code in gve_bldg (e.g. "BSS"), and gui_full_room_name is auto-generated from the full building name in Title Case plus the room number. A Hardware Options section below the counts edits dev_ settings that are not device counts (e.g. dev_relay_power, dev_screen_control), rendered from the UI schema.**
Devices	**One sub-tab per active device (Projector 1, Camera 2, …). Every field is rendered from the UI schema. The set of tabs is driven by the dev_ counts in SYSTEM_SETUP.**
System	**Every SYSTEM_SETUP property except the dev_ hardware counts, each rendered from the UI schema.**
Raw JSON	**A syntax-highlighted text editor of the whole config. Apply Changes parses it back into memory and also writes it to the working file.**
App Config	**Application paths (modules folder, buildings.json, processors.json, template config, ui_schema.json, key_map.json), the Theme Style and Text Size, the processor connection settings (SFTP username, port, and remote config path — Extron defaults), and the active deployment target. Reload buttons for the schema and key map live here.**

Theme Style. The App Config tab includes a Theme Style dropdown with two styles, each themed around an accent color picked from the swatch grid shown beneath the dropdown: Classic — the default — is a standard Material look, and Auris is a sci-fi HUD look with its own swatch set (amber, teal, magenta, and more). The sun/moon button on the toolbar still toggles dark and light within whichever style is selected, and every choice is saved and restored on the next launch. A Text Size dropdown below the theme settings scales all text in the app (Small 85% up to Huge 150%).

1.2 The supporting files you point the app at

In the App Config tab you can set explicit paths to several external files — but you no longer have to. **Every path that is left blank defaults to the Root Folder** (and the Root Folder itself defaults to the app's working directory). Drop all the files into one folder, point the Root Folder at it (or run the app from it), and everything resolves automatically:

File / path	Default when blank	Role
Root Folder	**App working directory**	**The base folder every other blank path resolves against.**
Template config.json	<root>/config.json	**Master template a new room is cloned from ("Create New Config"); also the audit baseline on load.**
ui_schema.json	<root>/ui_schema.json	**How each config key is shown in the GUI (label, description, widget type, options). Overlays the built-in defaults.**
key_map.json	<root>/key_map.json	**Legacy key-translation rules applied automatically on every load, before template migration.**
buildings.json	<root>/buildings.json	**Full building names → abbreviations; drives the building autocomplete and auto room name.**
processors.json	<root>/processors.json	**Deployment rooms/targets for the "Active Deployment Target" dropdown (supplies IP for SFTP). Read automatically on startup.**
Python modules path	<root>/devices	**Folder of Extron device .py modules, parsed for the keep-alive command and input dropdowns.**

First launch: the setup dialog

The very first time the app runs (no settings saved yet), a one-time setup dialog asks where each of these files is located. Pick the Root Folder first — every file found inside it turns green automatically — then Browse only for files stored elsewhere. Each choice is saved the moment it is picked. Finishing (or skipping) the dialog records the setup as complete, so the check is bypassed on every later launch; existing installs that already have saved paths are never asked. The same dialog can be re-opened any time with the "Run First-Time Setup" button at the top of App Config.

Resolution rules

An explicit path always wins. A blank path falls back to the file name shown above inside the Root Folder; a blank Root Folder falls back to the working directory. ui_schema.json and key_map.json additionally keep their old search (working directory, then next to the executable) when no root-folder copy exists. processors.json and buildings.json are read on boot from wherever they resolve. If a file is missing or invalid, the app logs it and falls back to built-in behavior — a broken or absent file can never take the editor down or block a config from loading.

2. How a config moves through the program

2.1 Getting a config into memory

There are three entry points:

- **Create New Config** — clones the template config, forces every `dev_` count to 0, and prunes all device blocks. This is the only time the template file is actually read into the editor.
- **Open Existing Config** — opens a local `.json` file and runs the full load pipeline (Section 2.2).
- **Download from Processor (SFTP)** — pulls `/config.json` off the processor, asks where to save the working copy, then runs the same load pipeline.

2.2 The load pipeline

Both "Open Existing" and "SFTP Download" run through one shared pipeline. The order matters and is worth knowing because it explains what the acknowledgement dialog shows you after a load:

1. **Key mapping** — `key_map.json` translates legacy section and property names to the current schema (Section 4). Runs first so everything downstream only sees current keys.
2. **Automatic backup** — the pristine original text is written next to the working file, e.g. `BSS103_old_config.json`. The backup name uses the room identity, which is why mapping runs first.
3. **Template migration** — any baseline `SYSTEM_SETUP` property missing from the file is injected with a safe default (Section 2.3).
4. **Building code normalization** — if `gve_bldg` holds a full building name found in `buildings.json`, it is converted to the building code (e.g. `BSS`) and logged as a BUILDING CODE entry in the acknowledgement dialog and the backup log.
5. **Device defaults fill** — when enabled in App Config (on by default), device blocks missing their type's `device_defaults` baseline properties (e.g. a DSP without its audio group numbers) get them added, one DEFAULTS entry per addition.
6. **Auto room name** — if `gve_bldg` resolves against `buildings.json`, `gui_full_room_name` is regenerated in Title Case.
7. **Device-count audit** — warns (never changes) when more device blocks exist than the `dev_` count allows; extra blocks are hidden and pruned on export.
8. **Template audit** — flags any section or property that is not in the template config, so leftovers and custom keys are visible.
9. **Logs** — every change is written to the migration audit log and, when anything changed, to a per-load change log next to the backup. The same list is shown in the on-screen "Legacy Config Updated" dialog.

2.3 Baseline SYSTEM_SETUP defaults injected on load

These values are schema-editable: the SYSTEM_SETUP defaults come from the top-level "system_defaults" object in ui_schema.json (defining it replaces the built-in set), and the dev_ hardware counts are always injected as "0" for every family listed in "device_types".

These are hard-coded safety defaults in the migration step (independent of the schema) so the current template can always function. They are only added when *missing*; existing values are never overwritten:

```
gve_bldg: "UNKNOWN"          dev_projectors: "0"      gui_mic_mix: "No"
gve_room: "000"              dev_cameras: "0"         gui_routing_available: "No"
gui_full_room_name:          dev_switchers: "0"       gui_routing_mode: "Normal"
  "Legacy Room Update"       dev_dsps: "0"            gui_tab: "2_Cam_Dev"
                             dev_usb_switchers: "0"   gui_capture_source_available: "No"
                             dev_media_ports: "0"     gui_usb_or_vga: "USB"
                             dev_wireless: "0"
                             dev_recorders: "0"
                             dev_screens: "0"
                             dev_power_controllers: "0"
```

2.4 Getting a config back out

- **Export Config Locally** — saves a pruned `config.json` via a save dialog, default-named from the building abbreviation and room (e.g. `BSS_103_config.json`).
- **Upload to Processor (SFTP)** — pushes the pruned config to `/config.json` on the processor, optionally alongside an extra file such as `Whereused.csv`.
- **Sorted keys** — every output path (Apply Changes in the Raw JSON tab, Export, and SFTP upload) writes the config with keys sorted alphabetically in natural order, so `PROJECTORDEVICE_2` comes before `PROJECTORDEVICE_10`. The Raw JSON view always displays this same sorted form.
- **Raw JSON → Apply Changes** — also writes straight to the working file, so Apply doubles as Save.

> **Pruning vs. the on-disk working file**
>
> Export and SFTP upload always write the *pruned* config (device blocks numbered above their dev_ count are dropped). The Raw JSON view also renders pruned, so what you see matches what ships. The working-file save from Apply keeps the full un-pruned config so nothing is lost mid-edit.

3. Setting up ui_schema.json

The UI schema is the file you will touch most often. It is a single JSON object with a top-level `"fields"` object; each entry is keyed by a `config.json` property name and describes how that property is rendered on the System and Device tabs. The same schema serves both — there is no separate "system schema" and "device schema".

3.1 File skeleton

```
{
  "schema_version": 1,
  "__readme": [ "comments ... keys starting with __ are ignored" ],
  "fields": {
    "com_type": {
      "type": "dropdown",
      "label": "Connection Type",
      "description": "Text behind the (i) info button.",
      "helperText": "Small grey hint under the field.",
      "options": ["Serial", "SerialOverEthernet", "Network"]
    }
  }
}
```

3.2 The properties every field entry supports

Property	Meaning
`label`	**Field label shown in the editor. Defaults to the raw config key.**
`description`	**Text behind the (i) info button. Falls back to the built-in dictionary if omitted.**
`helperText`	**Small grey hint line under the field.**
`type`	**auto | text | int | double | bool | dropdown | combo | hidden. See 3.3.**
`options`	**For dropdown / combo. Plain strings, or objects with value/label (and values for combo).**
`writes`	**For combo only — the list of config keys the single dropdown writes to.**
`moduleCommand`	**For module_states only — the command in the device's Python module whose states become the options (e.g. "Input").**
`addIfMissing`	**device_fields only — render the field on matching device tabs even when the key does not exist in that device block yet; the first edit writes it into config.json.**

3.3 Field types

type	Renders as	Notes
`auto`	**Inferred widget**	**Default when type is omitted. bool value → switch, int → numeric field, double → numeric field, anything else → text field.**
`text`	**Free text field**	**Stores a string.**
`int`	**Numeric field**	**Parses and stores an integer, keeping config.json numeric.**
`double`	**Numeric field**	**Parses and stores a double.**
`bool`	**On/off switch**	**Stores true/false.**

type	Renders as	Notes
dropdown	**Fixed dropdown**	**Uses the options array. A stored value not in the list is shown flagged rather than silently dropped.**
combo	**One dropdown → many keys**	**Writes several keys at once via writes + per-option values. See 3.5.**
hidden	**Not rendered**	**For keys managed by a combo or the wizard (e.g. gui_tab_type).**
module_states	**Autocomplete (module-driven)**	**Options parsed live from the device's selected Python module: the states of the command named by moduleCommand (AllowedValues, falling back to the Set/Match state dictionaries). Manual typing is always allowed; unresolved modules fall back to free text.**

3.4 Dropdown options: two forms

Plain strings, when the stored value and the shown label are identical:

```
"options": ["TCP", "SSH", "UDP"]
```

Objects, when you want a friendlier label than the stored value:

```
"options": [
  { "value": "Yes",     "label": "Yes (Single Mic w/ Ducking)" },
  { "value": "No",      "label": "No (Single Mic w/ Mute)" },
  { "value": "Ceiling", "label": "Ceiling (Voicelift & Mute)" }
]
```

3.5 Combo fields (one dropdown that writes several keys)

A combo lets one dropdown set two or more config keys together. The stock example is `gui_inputs` + `gui_tab_type`, where a single "Sources & Routing" choice writes both. Declare the keys in `writes`, and give each option a `values` array with one entry per key, in order:

```
"gui_inputs": {
  "type": "combo",
  "label": "Sources & Routing Config (gui_inputs + gui_tab_type)",
  "writes": ["gui_inputs", "gui_tab_type"],
  "options": [
    { "values": ["3", "WL"],         "label": "3 Sources: PC, HDMI, & Wireless" },
    { "values": ["4", "DOC_USB"],    "label": "4 Sources: PC, HDMI, Doc Cam, & USB" },
    { "values": ["5", "DOC_USB_WL"],"label": "5 Sources: PC, HDMI, Doc Cam, USB, &
Wireless" }
  ]
},
"gui_tab_type": { "type": "hidden" }
```

The current selection is identified by joining the written values with an underscore (e.g. `5_DOC_USB_WL`). Mark every key that a combo writes but that should not appear on its own as `"type": "hidden"`, or it will show up twice.

3.6 Wildcards for families of keys

A key may contain a `*` so one entry covers a whole family. Exact keys always win over wildcards, and a more specific wildcard wins over a broader one:

```
"power1_outlet_*":        { "description": "APC outlet name. 23 char limit.",
                            "helperText": "Max 23 characters" },
"power1_outlet_*_action": { "description": "Restrict this outlet to a reboot action." },
"relay_port_*":           { "description": "Screen-control relay port (e.g. RLY1)." },
"group_*":                { "description": "Audio group number on the switcher/DSP." }
```

Here `power1_outlet_3` matches the first entry, `power1_outlet_3_action` matches the more specific second entry, and any exact `group_prog_gain` entry would beat the `group_*` fallback.

3.7 Unknown keys and the fallback chain

When a config key has no schema entry and no built-in dictionary description, the field still renders — as a plain text field with a red outline and a warning icon telling you to add it to `ui_schema.json`. Nothing is hidden or lost; the red styling is a maintenance prompt. Descriptions resolve in this order:

1. The `description` in the schema entry.
2. The built-in `ConfigDictionary` description (legacy, compiled in).
3. None → the field is treated as unknown and flagged.

3.8 Adding a brand-new key: the template block

The shipped `ui_schema.json` includes a copy-paste template. Duplicate it, rename it to your real key (dropping the leading `__`), then press **Reload Schema**. It appears wherever that key exists in the loaded config:

```
"__example_new_key": {
  "label": "My New Feature",
  "type": "dropdown",
  "options": ["Enabled", "Disabled"],
  "description": "Explain what this new key does here."
}
```

3.9 Device-type fields (device_fields)

Next to `"fields"`, the schema supports a top-level `"device_fields"` object holding entries that apply only to matching device sections, keyed by a section pattern such as `"PROJECTORDEVICE_*"`. Scoped entries use exactly the same properties as `"fields"` and win over the global entries on those tabs. Combined with `addIfMissing`, this lets a device-type-only setting ship purely by editing `ui_schema.json`. Example — projector input options read from each module's own Input command dictionary:

```
"device_fields": {
  "PROJECTORDEVICE_*": {
    "input": {
      "type": "module_states",
      "moduleCommand": "Input",
      "addIfMissing": true,
      "description": "Options come from the module's Input command."
    }
  }
}
```

When a device type overrides `input` this way, the built-in input autocomplete (which scans the module for common input strings) is replaced by the schema-driven field for those tabs only.

3.10 Device defaults (device_defaults)

Next to `"device_fields"`, a top-level `"device_defaults"` object maps section patterns to property/value pairs that are merged into every newly created device block of that type when the Setup Wizard counts change. Existing/template values are never overwritten — only missing keys are added — and exact section names (e.g. `"SWITCHERDEVICE_1"`) override wildcard patterns per property. The shipped schema gives projectors `input` and `relay_host`, DSPs their eleven audio group numbers, and switcher 1 its audio groups. Edit the values and press Reload Schema — no recompile. The same defaults can also be filled into loaded configs: the "Fill missing device defaults when loading" switch in App Config (on by default) adds them during the load pipeline, reported in the acknowledgement dialog.

```
"device_defaults": {
  "PROJECTORDEVICE_*": { "input": "", "relay_host": "processor1" },
  "DSPDEVICE_*": { "group_prog_gain": "1", "group_prog_mute": "5", "...": "..." },
  "SWITCHERDEVICE_1": { "group_mic_gain": "10", "group_prog_gain": "1", "...": "..." }
}
```

3.11 Device types (device_types)

The device families the Setup Wizard manages come from the top-level `"device_types"` object: each `SYSTEM_SETUP dev_` count key mapped to its section prefix (required) and wizard label. This list REPLACES the built-in one, so keep every family you want listed; its order is the wizard's display order. An added family automatically gets a wizard count dropdown, device tabs, export pruning, count audits, and a "0" default injected on load. Pair it with `fields` / `device_fields` entries for its properties and `device_defaults` for its baseline values. Each family also supports optional "max" (the highest count the wizard offers, default 8), "keepAlivePreference" (keep-alive command names tried in order when a new device's module is parsed), and "template" (the full block used for new devices when the config has no <PREFIX>1 block to copy).

```
"device_types": {
  "dev_projectors": { "prefix": "PROJECTORDEVICE_", "label": "Projectors / Displays" },
  "dev_lifts":      { "prefix": "LIFTDEVICE_",      "label": "Projector Lifts" },
  "...": {}
}
```

4. System setup vs. device setup

In the `config.json`, there are two structural kinds of top-level blocks, and the schema handles them the same way but the app routes them to different tabs.

4.1 How the app decides what is "system" and what is a "device"

	System	Devices
Config block	`SYSTEM_SETUP`	`PROJECTORDEVICE_1, CAMERADEVICE_2, …`
Shown on	**System tab**	**Devices tab (one sub-tab per active device)**
Which keys appear	**Every SYSTEM_SETUP key except the dev_ counts (those are owned by the Wizard)**	**Every key inside each active device block**
How many blocks	**Exactly one**	Driven by the dev_ count for that family in SYSTEM_SETUP
Schema source	**The same fields object**	**The same fields object**

The key point: **a device tab shows every property inside that device block, and each property is matched against the schema by its own name** — exactly the same lookup the System tab uses. So `ip_address` on a projector and `ip_address` somewhere in SYSTEM_SETUP both use the single `"ip_address"` schema entry. You do not (and cannot) write a separate schema per device type; you write one entry per *property name*, and it applies everywhere that property occurs.

4.2 Practical consequence for schema authoring

- A property used *only* in SYSTEM_SETUP (e.g. `gui_routing_mode`, `ntp_primary`) — add one entry; it renders on the System tab.
- A property used *only* in device blocks (e.g. `keep_alive_interval`, `module`) — add one entry; it renders on every device tab that has that key.
- A property shared by both (e.g. `gve_id`, `host`) — still just one entry; it renders in both places identically.
- Per-outlet / per-relay / per-group families inside device blocks — use a wildcard entry (`power1_outlet_*`) so you do not repeat yourself eight times.

4.3 Controlling device count and tabs

You do not create device tabs in the schema. The **Wizard tab** sets a `dev_` count (e.g. `dev_projectors = 2`), which generates `PROJECTORDEVICE_1` and `PROJECTORDEVICE_2` blocks and their tabs. The schema only governs what the fields *inside* each block look like. Changing a count regenerates the blocks (resetting them from the template), so set counts before fine-tuning device fields.

> **The dev_ counts are the source of truth for devices**
> A device block only shows up as a tab if its number is within the dev_ count. Blocks above the count are hidden and pruned on export. If a migrated room has four projector blocks but `dev_projectors` is "1", the extra three are flagged in the load dialog and will be dropped unless you raise the count in the Wizard.

5. Setting up key_map.json

Where the UI schema controls *display*, the key map controls *translation* of older files. It runs automatically at the very start of every load, turning legacy names like `CAMERA1DEVICE.IPADDRESS` into current names like `CAMERADEVICE_1.ip_address` before anything else sees the config. With no file present, mapping is a no-op and loading behaves exactly as it always did.

5.1 Processing order

The mapper applies its rule groups in this fixed order. Later steps depend on earlier ones (e.g. moves target already-renamed sections):

1. sections — rename top-level blocks
2. properties — explicit per-property renames
3. auto_case_normalization — automatic case/underscore renames
4. value_map — normalize stored values
5. moves — relocate a property into another section
6. remove_unused_devices — drop legacy blocks not in use
7. escape_carriage_returns — convert control characters to the \\r file marker
8. device_labels / device_templates — smart name / btn / lbl / module fill-in
9. companion_keys — create matching partner keys (e.g. outlet _action keys)
10. defaults — inject any still-missing keys

5.2 The rule groups

sections — rename top-level blocks

A list of `{ match, rename_to }`. `match` is a whole-name regex; `$1..$9` insert capture groups. First matching rule wins.

```
"sections": [
  { "match": "CAMERA(\\d+)DEVICE", "rename_to": "CAMERADEVICE_$1" },
  { "match": "CAMERADEVICE",       "rename_to": "CAMERADEVICE_1" },
  { "match": "SYSTEM|SYSTEMSETUP|SYSTEM SETUP", "rename_to": "SYSTEM_SETUP" }
]
```

properties — explicit renames

A flat `OLDNAME → new_name` map applied inside every section. These run first and always win over auto-normalization.

```
"properties": {
  "COMTYPE": "com_type",
  "IPADDRESS": "ip_address",
  "PowerOutlet1": "power1_outlet_1",
  "gui_TABTYPE": "gui_tab_type",
  "dev_DSP": "dev_dsps"
}
```

auto_case_normalization

When `true`, any legacy property whose lowercased, underscore-stripped form matches a known current key is renamed automatically — so `Output_Proj1 → output_proj_1`, `GVE_BLDG → gve_bldg`, `COMTYPE → com_type` need no explicit entry. The "known current keys" are drawn from the UI schema plus the built-in dictionary. Only spellings that differ *structurally* (not just case) need an explicit `properties` entry.

value_map — normalize values

Keyed by the *new* property name (after rename). Mapped values keep their JSON type, so a string can become a boolean or a differently-typed string:

```
"value_map": {
  "active_notifications": { "True": true, "False": false },
  "dev_dsps":  { "Yes": "1", "No": "0" },
  "com_type":  { "NETWORK": "Network", "SERIAL": "Serial" },
  "protocol":  { "TCP/IP": "TCP", "tcp": "TCP", "ssh": "SSH" }
}
```

moves — relocate a property to another section

Legacy files kept per-device GVE IDs inside `SYSTEM`; the current schema stores them on each device. A move rule relocates them, with capture groups usable in the target section name:

```
"moves": [
  { "from_section": "SYSTEM_SETUP",
    "key_match": "GVE_ID_Camera_(\\d+)",
    "to_section": "CAMERADEVICE_$1",
    "to_key": "gve_id" }
]
```

remove_unused_devices + device_counts

When `remove_unused_devices` is `true`, legacy device blocks whose family count in `SYSTEM_SETUP` says they are not in use (No / 0, or numbered above the count) are *removed* rather than converted. `device_counts` maps each count key to its section prefix. Crucially, only blocks that were **renamed from a legacy name** in step 1 are eligible — new-style template files that deliberately carry every block keep them all.

```
"remove_unused_devices": true,
"device_counts": {
  "dev_cameras": "CAMERADEVICE_",
  "dev_projectors": "PROJECTORDEVICE_"
}
```

escape_carriage_returns

When `true`, real CR/CRLF control characters inside string values are converted to the new-style marker the processor code reads: the **saved config.json literally contains** `\\r` (two backslashes + r). A lone newline is left untouched. For example, the power-outlet GUI names come out of a conversion looking like this on disk:

```
"power1_outlet_4": "TLP\\rPoE",
"power1_outlet_6": "Doc\\rCam",
"power1_outlet_8": "USB\\rSwitch"
```

> **Why the code says \r but the file says \\r**
>
> There are two escaping layers. In memory, the app stores the two-character sequence backslash+r. When the config is written, the JSON encoder escapes that backslash — so the file on disk shows `\\r`. When the processor later parses the JSON, that decodes back to the single backslash+r marker its GUI code uses. The two representations are the same value at different layers; the file always carries the doubled form.

companion_keys — auto-create matching partner keys

A list of rules that guarantee a partner key exists for every property matching a pattern, **with the number carried over from the matched key**. The stock use: every converted `power1_outlet_N` gets a matching `power1_outlet_N_action` set to `null` — the companion is only created for outlet numbers that actually exist, and an existing value (e.g. an outlet already set to `"Reboot"`) is never overwritten:

```
"companion_keys": [
  {
    "key_match": "power1_outlet_(\\d+)",
    "ensure_key": "power1_outlet_$1_action",
    "value": null
  }
]
```

`key_match` is a whole-key regex; `$1..$9` capture groups may be used in `ensure_key`. An optional `"section"` wildcard (e.g. `"POWERDEVICE_*"`) limits the rule to matching sections. Companions run after all renames and smart defaults, right before the generic `defaults` injection, so they always see the converted key names.

device_labels + device_templates — smart fill-in

These generate friendly device data when converting legacy files, filling only keys that are missing:

- `device_labels` — prefix → friendly label, used to build `name` as "*Label - model*" (numbered only when the family has more than one unit, e.g. "Camera 2 - Cam570").
- `device_templates` — reference blocks from the current template. `btn_name`/`lbl_name` carry over by device *number*; `module` and `keep_alive_trigger` carry over when the legacy `model` matches a template block's model.

defaults — inject missing keys

After everything else, for each section matching a `SECTIONPATTERN_*` wildcard, any listed key still missing is injected. `{n}` in a string value becomes the device number. Existing values are never overwritten:

```
"defaults": {
  "CAMERADEVICE_*": {
    "btn_name": "Btn_Con_Cam{n}",
    "host": "processor1",
    "gve_id": "Cam{n}",
    "keep_alive_command": "Power",
    "keep_alive_interval": 10,
    "protocol": "UDP"
  }
}
```

> **Comment keys**
>
> Anywhere in either file, any key beginning with __ (e.g. `__comment`, `__readme`) is ignored by the parser, so you can annotate freely.

6. Putting it together: a setup checklist

6.1 First-run configuration

1. Simplest setup: put everything in one folder — `config.json` (template), `processors.json`, `buildings.json`, `ui_schema.json`, `key_map.json`, and a `devices` sub-folder holding the Extron Python modules.
2. On the very first launch, the **setup dialog** opens automatically. Pick your folder as the Root Folder — every file it finds turns green — and Browse for anything stored elsewhere. Press Finish; the dialog never appears again (*and everything, including processors.json, loads on every startup from then on*). The dialog also asks for your preferred accent color — the Classic theme (the default) is built around it, and it can be changed any time under App Config > Theme Style.
3. To adjust later: use the individual path fields in App Config (an explicit path always overrides its default), or press **Run First-Time Setup** to reopen the guided dialog.
4. Confirm the App Config panel reports the active schema source and field count — that tells you the file loaded rather than falling back to built-in defaults.

6.2 Adding support for a new config property

1. Add the property to the template `config.json` (so new rooms include it and the template audit does not flag it).
2. Add a `fields` entry in `ui_schema.json` with a label, description, type, and any options.
3. If older files spell it differently, add a `properties` rename (or rely on auto-normalization if it is only a case/underscore difference) in `key_map.json`.
4. Press **Reload Schema** and **Reload Key Map** in App Config. The field now appears wherever the key exists — System tab or device tabs — with no rebuild.
5. For a property that only one device type should have, put the entry under `device_fields` with the device pattern (e.g. `"PROJECTORDEVICE_*"`) instead of `fields`, and set `addIfMissing: true` so it appears on those tabs right after Reload Schema.

6.3 Quick reference: which file does what

I want to…	Edit
Change a field's label, help text, or info description	`ui_schema.json`
Add a device-type-only field, or options read from the Python module (device_fields / module_states)	`ui_schema.json`
Turn a text field into a dropdown / switch / numeric field	`ui_schema.json (type + options)`
Make one control set several keys at once	`ui_schema.json (combo)`
Cover a family like power1_outlet_1..8 with one entry	`ui_schema.json (wildcard *)`
Rename an old block or property on load	`key_map.json (sections / properties)`
Normalize old stored values (Yes→1, True→true)	`key_map.json (value_map)`
Move a legacy SYSTEM field onto its device	`key_map.json (moves)`
Auto-create matching outlet _action keys (null)	`key_map.json (companion_keys)`
Drop dead legacy device blocks automatically	`key_map.json (remove_unused_devices)`

I want to…	Edit
Fill missing device keys with sane defaults	`key_map.json (defaults)`
Change how many device tabs exist	Wizard tab (dev_ counts)
Change where every blank path defaults to	App Config — Root Folder

End of guide.